

# 深層神經網路訓練

## 梯度穩定與遷移學習

書本第 11 章（上） | 含第 10 章必要的 PyTorch 銜接

## 學習重點與成果

---

- 讀懂 PyTorch 訓練步驟，解釋梯度與初始化。
- 比較激活函數，手算 BatchNorm 與 LayerNorm。
- 梯度裁剪、遷移學習與實驗設計。

成果：能指出訓練流程的錯誤，並用證據選擇改善方法。

「選讀」頁提供進階推導與實作補充。程式以讀碼為主。

 **配套 Notebook**： `programs/notebooks/12_深層神經網路訓練_梯度與遷移.ipynb`。本檔提供公式驗算、機制實驗與結果圖；框架片段及完整模型案例另依頁面說明閱讀。

## 課程位置：ANN、PyTorch 與深層訓練

---

| 書本章節          | 問題               | 本課銜接              |
|---------------|------------------|-------------------|
| Ch.9 人工神經網路   | 網路如何表示函數、計算梯度？   | 已有 11_人工神經網路入門.md |
| Ch.10 PyTorch | 如何建立模型、迭代訓練與評估？  | 先補本章範例需要的操作       |
| Ch.11 深層訓練    | 加深以後，如何穩定、加速與泛化？ | 本份及下一份的主線         |

先建立激活函數與 Dropout 的基本概念；本章補上原理與操作。

## PyTorch 銜接：資料形狀與角色

本章資料為 Fashion MNIST：28×28 灰階影像、10 個互斥類別。

| 物件                          | 形狀／型態                                      | 角色           |
|-----------------------------|--------------------------------------------|--------------|
| <code>X_batch</code>        | <code>[B, 1, 28, 28]</code> 浮點數            | 前處理後的影像      |
| <code>y_batch</code>        | <code>[B]</code> ， <code>torch.long</code> | 0-9 的類別索引    |
| <code>model(X_batch)</code> | <code>[B, 10]</code> 浮點數                   | 未正規化的 logits |
| <code>DataLoader</code>     | 逐批回傳 <code>(X_batch, y_batch)</code>       | 訓練可打散，驗證固定   |

訓練集學參數，驗證集選設定，測試集留到最後。

## PyTorch 銜接：最小分類模型

---

```
import torch
from torch import nn

device = torch.device("cpu") # 讀碼與小型示範可用 CPU
model = nn.Sequential(
    nn.Flatten(), nn.Linear(784, 100), nn.ReLU(),
    nn.Linear(100, 10)
).to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
```

CrossEntropyLoss 直接接 logits，模型末端不另加 Softmax。

## 選讀 | 輸出層與損失函數要配對

---

延伸練習：在本章 Notebook 對照分類損失、輸出層與標籤編碼。

```
# 程式片段：最後一層與訓練設定
keras.layers.Dense(10, activation="softmax")
model.compile(loss="sparse_categorical_crossentropy",
               optimizer="sgd", metrics=["accuracy"])
```

機率輸出需配對交叉熵；PyTorch 的 logits 輸出配對 `CrossEntropyLoss`。

換框架時，要連同輸出層、損失函數與標籤格式一起檢查。

## PyTorch 銜接：一次批次更新

---

```
model.train()
for X_batch, y_batch in train_loader:
    X_batch = X_batch.to(device)
    y_batch = y_batch.to(device)
    optimizer.zero_grad(set_to_none=True)
    logits = model(X_batch)
    loss = loss_fn(logits, y_batch)
    loss.backward()
    optimizer.step()
```

`backward()` 計算並累積梯度，`step()` 才更新參數。

## PyTorch 銜接：驗證與訓練模式

---

```
model.eval()
loss_sum, n = 0.0, 0
with torch.no_grad():
    for X_batch, y_batch in valid_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        loss = loss_fn(model(X_batch), y_batch)
        loss_sum += loss.item() * len(y_batch)
        n += len(y_batch)
val_loss = loss_sum / n
```

`eval()` 切換 BN／Dropout 行為，`no_grad()` 關閉梯度紀錄。

## 讀碼檢核：哪一步需要修正？

---

1. 在 `CrossEntropyLoss` 前先做 `Softmax`。
  2. 每批只呼叫 `backward()`，整堂課都沒清除梯度。
  3. 用 `no_grad()` 驗證，卻忘記 `model.eval()`。
  4. 每個 epoch 用測試 accuracy 決定是否換模型。
- 兩人一組，各選一項說明「會發生什麼」及「如何改」。

## 選讀 | 手刻 MLP：前向、反向與參數更新

---

閱讀手刻 MNIST 的 `train()` 更新段：

```
# 程式片段：上方已完成 forward() 與 backward()  
model.weight_h -= learning_rate * d_loss__d_w_h  
model.bias_h -= learning_rate * d_loss__d_b_h  
model.weight_out -= learning_rate * d_loss__d_w_out  
model.bias_out -= learning_rate * d_loss__d_b_out
```

四行就是基本 SGD 更新，對照本課的 `optimizer.step()`。

若將輸出層從 Sigmoid 改為 Softmax，也必須依新的損失函數重推反向公式。

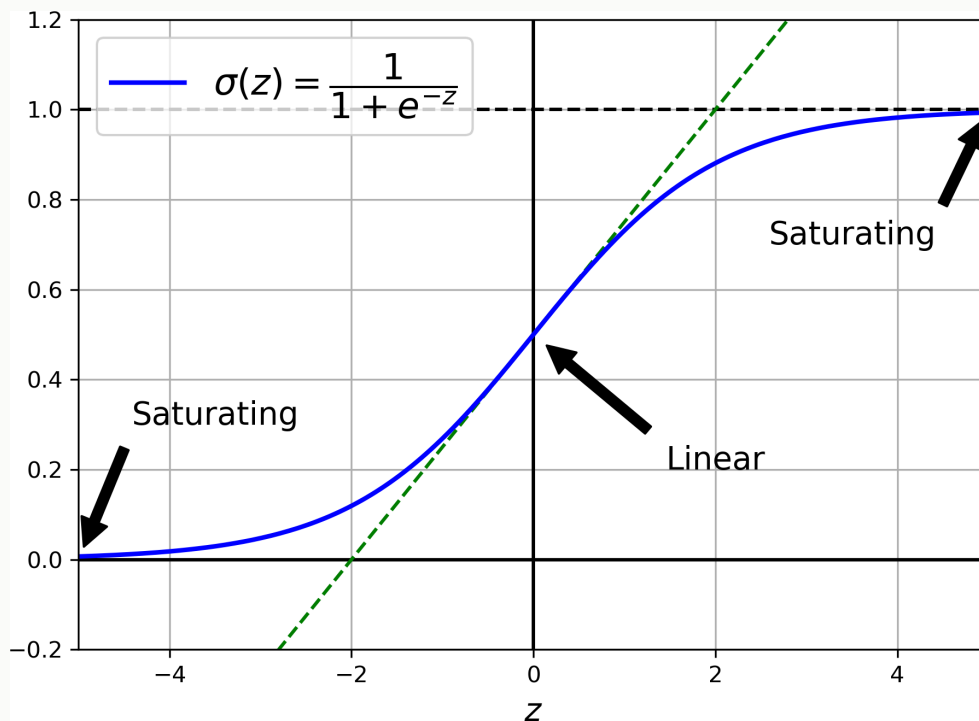
## 深層網路的四個訓練困難

---

- **梯度不穩定**：前面層學不到，或參數更新突然失控。
- **標註資料不足**：大模型很快記住少量訓練樣本。
- **訓練太慢**：有梯度，卻需要很多次更新。
- **泛化不足**：訓練誤差下降，驗證誤差反而上升。

先判斷問題類型，再選方法；同一現象可能有多種原因。

## 圖 11-1：Sigmoid 飽和與小梯度



輸入落在曲線兩端時，輸出幾乎不變，導數接近 0。

**先估計：**即使每個導數都取最大值  $1/4$ ，十個連乘後會比單層小多少？

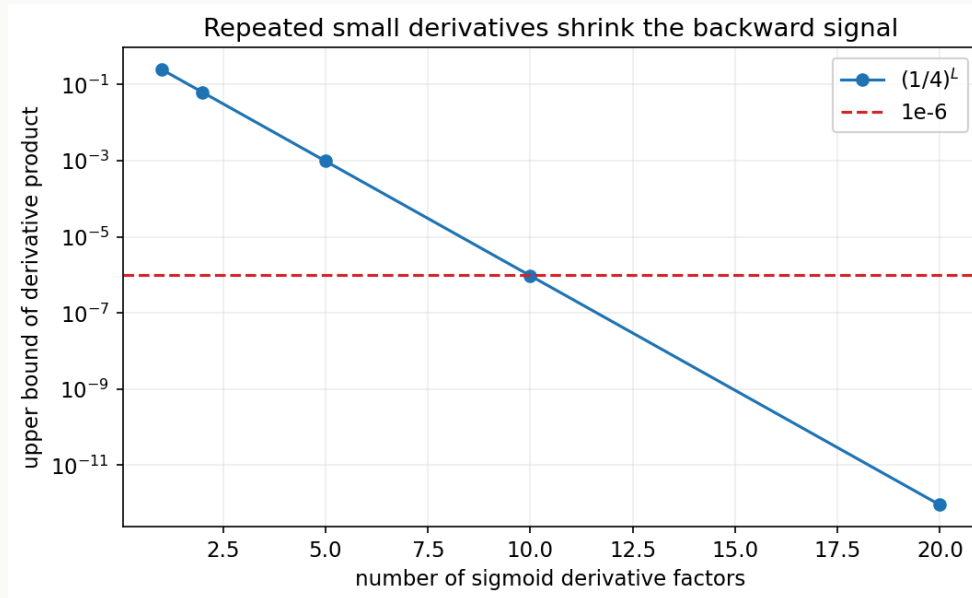
# 用導數連乘核對反向訊號的縮小

$$\delta^{(l)} = \left( \delta^{(l+1)} W^{(l+1)T} \right) \odot \phi'_l(Z^{(l)})$$

反向傳播反覆乘上權重與激活導數；先只計算 Sigmoid 導數的上界：

```
depth = np.array([1, 2, 5, 10, 20])  
upper = 0.25 ** depth
```

十個因子約為  $9.54 \times 10^{-7}$ 。圖的縱軸為連乘上界（對數尺度），不是實測網路梯度。



完整梯度仍取決於權重、分支與資料；本例只核對導數縮小效應。

參考程式：[programs/notebooks/12\\_深層神經網路訓練\\_梯度與遷移.ipynb](#)

表 11-1：初始化尺度與激活函數

$\text{fan}_{in}$  是輸入數， $\text{fan}_{out}$  是輸出數。

| 初始化           | 書中配對                    | 常態分布的變異數 $\sigma^2$                      |
|---------------|-------------------------|------------------------------------------|
| Glorot/Xavier | 線性、tanh、sigmoid、softmax | $2/(\text{fan}_{in} + \text{fan}_{out})$ |
| He/Kaiming    | ReLU 及書中列出的變體           | $2/\text{fan}_{in}$                      |
| LeCun         | SELU                    | $1/\text{fan}_{in}$                      |

若用均勻分布  $U(-r, r)$ ，則  $r = \sqrt{3\sigma^2}$ 。

## 初始化手算：100 個輸入、50 個輸出

| 方法     | $\sigma^2$     | 標準差 $\sigma$ |
|--------|----------------|--------------|
| Glorot | $2/(100 + 50)$ | ?            |
| He     | $2/100$        | ?            |
| LeCun  | $1/100$        | ?            |

如果把同一個隨機值填滿整層權重，是否已經打破對稱？

## PyTorch：隱藏層與輸出層分開初始化

---

```
# 使用前面建立的 784→100→10 模型
nn.init.kaiming_normal_(model[1].weight, nonlinearity="relu")
nn.init.zeros_(model[1].bias)
nn.init.xavier_normal_(model[3].weight)
nn.init.zeros_(model[3].bias)
```

隱藏層接 ReLU，輸出層提供 logits，兩者的角色不同。

不要在每個 epoch 重做初始化，否則會清除已學到的參數。

## 選讀 | 手刻 MLP 的初始化尺度

---

閱讀 `NeuralNetMLP.__init__` 的初始化設定：

```
# 程式片段：scale 是標準差
rng = np.random.RandomState(random_seed)
self.weight_h = rng.normal(
    loc=0.0, scale=0.1, size=(num_hidden, num_features))
self.bias_h = np.zeros(num_hidden)
```

原模型 784 個輸入、50 個隱藏單元，固定  $\sigma = 0.1$ 。

Glorot 對此層給  $\sigma = \sqrt{2/(784 + 50)} \approx 0.049$ 。

兩者不同：初始化尺度應連同激活函數、層寬與深度比較。

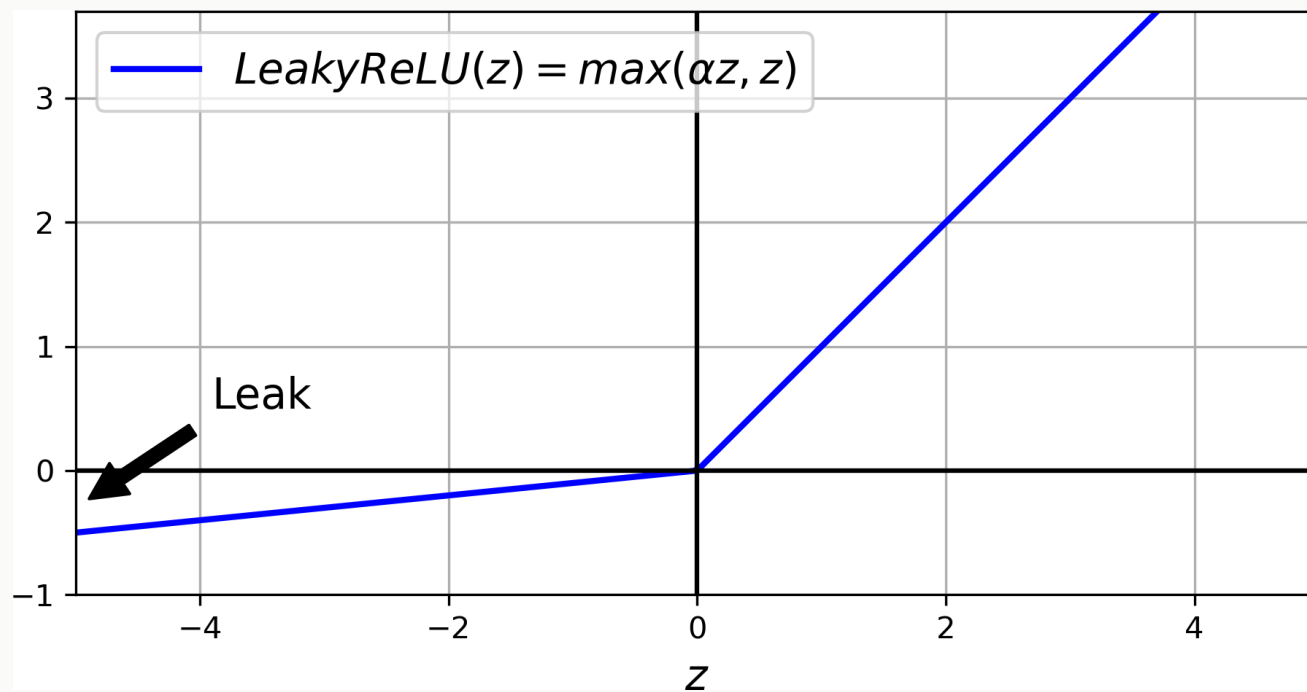
## ReLU 能完全避免梯度問題嗎？

---

不能。ReLU 在正半軸的導數為 1，能減輕部分梯度消失問題；負半軸導數為 0，神經元若長期落在負半軸，可能停止更新。

Leaky ReLU 在負半軸保留小斜率；初始化、正規化與學習率也會影響梯度穩定性。

## 圖 11-2：Leaky ReLU 保留負半軸斜率



$\text{LeakyReLU}(z) = \max(\alpha z, z)$ ，通常  $0 < \alpha < 1$ 。

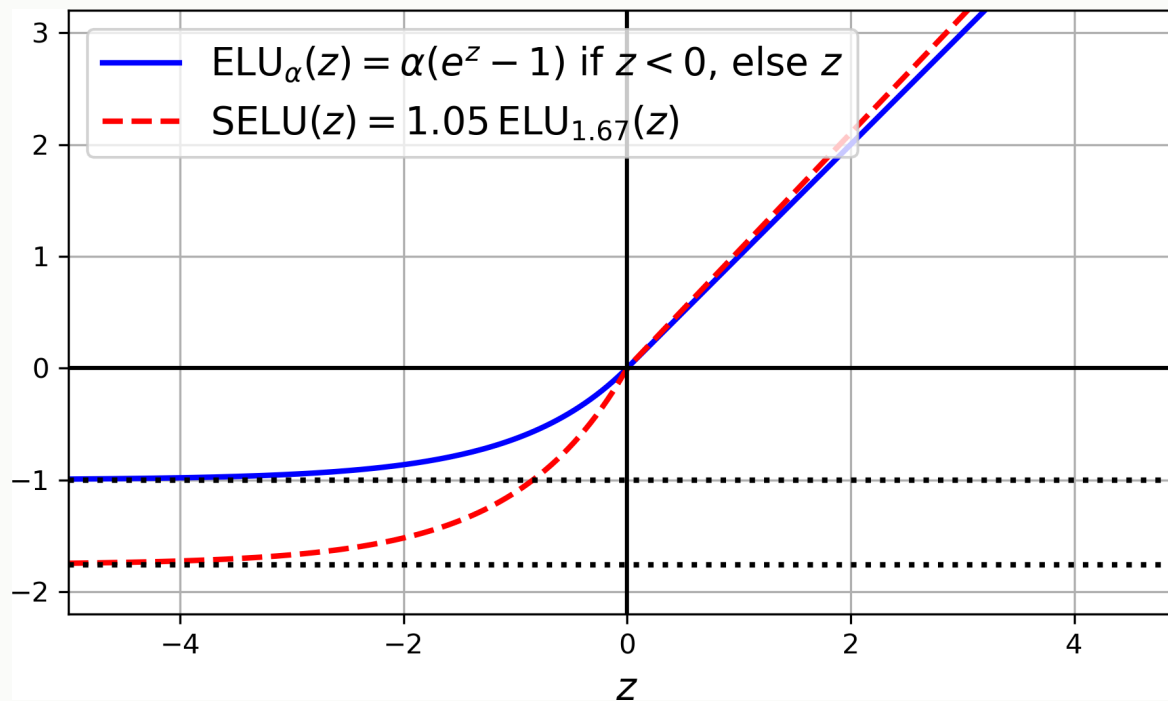
## ReLU 變體與初始化配對

| 激活         | 負半軸怎麼處理？      | PyTorch                        |
|------------|---------------|--------------------------------|
| ReLU       | 設為 0          | <code>nn.ReLU()</code>         |
| Leaky ReLU | 固定斜率 $\alpha$ | <code>nn.LeakyReLU(0.2)</code> |
| PReLU      | 斜率也是待學參數      | <code>nn.PReLU()</code>        |
| RReLU      | 訓練時抽樣斜率，推論固定  | <code>nn.RReLU()</code>        |

Leaky ReLU 的 He 變異數可調為  $2/[(1 + \alpha^2)\text{fan}_{in}]$ 。

比較 Leaky ReLU 在 `alpha=0.05` 與 `0.2` 時的函數值與導數。

## 圖 11-3：ELU 與 SELU



ELU 允許負值並逐漸飽和；SELU 再調整整體尺度。

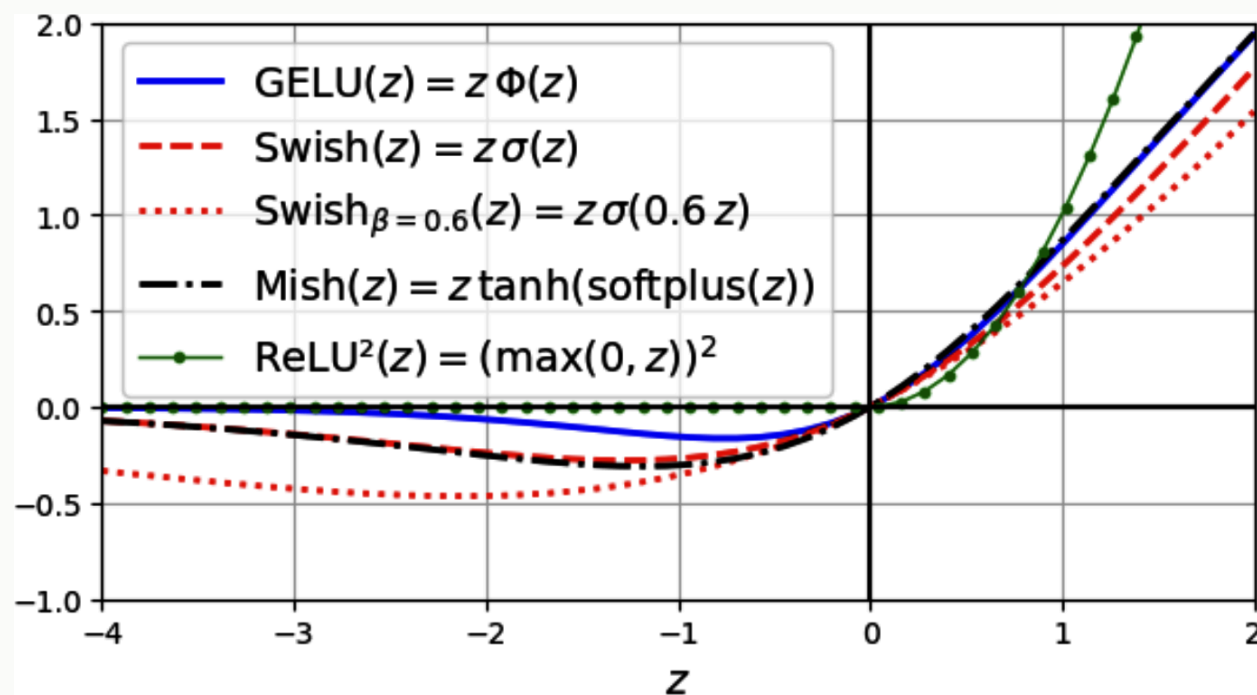
## SELU 自我正規化的條件

---

- 使用連續堆疊的全連接隱藏層，並採用 SELU。
- 輸入先標準化，隱藏權重使用 **LeCun normal**。
- 避免任意混入 BN、LN 或一般 Dropout 後仍宣稱自我正規化。
- 需要 Dropout 時，搭配保留分布特性的 `nn.AlphaDropout`。

這是一組有條件的設計，不能只把 ReLU 名稱改成 SELU。

## 圖 11-4：平滑激活函數的差異



比較負半軸、零點附近與大正值區域；曲線不同會改變梯度。

## GELU、SiLU、Mish 與 ReLU<sup>2</sup>

| 函數                         | 定義                       | 實作／提醒                                     |
|----------------------------|--------------------------|-------------------------------------------|
| GELU                       | $z\Phi(z)$               | <code>nn.GELU()</code> ， $\Phi$ 為標準常態 CDF |
| SiLU／Swish ( $\beta = 1$ ) | $z\sigma(z)$             | <code>nn.SiLU()</code>                    |
| Mish                       | $z \tanh(\log(1 + e^z))$ | <code>nn.Mish()</code>                    |
| ReLU <sup>2</sup>          | $\max(0, z)^2$           | 正值越大，導數也越大                                |

比較驗證表現與運算成本；較平滑不等於一定較準。

## 選讀 | 激活函數：把公式對到程式碼

閱讀激活函數程式片段，假設已匯入 NumPy 與 `erf`。

```
# 程式片段：保留函數之間的呼叫關係
def gelu(x):
    return 0.5 * x * (1 + erf(x / np.sqrt(2)))
def swish(x):
    return x * sigmoid(x)
def mish(x):
    return x * tanh(softplus(x))
```

對照同檔的導數，在  $x = -2, 0, 2$  判斷哪些位置仍能傳遞梯度。

## 選讀 | SwiGLU 的形狀與門控

---

將線性層輸出分成兩半，逐元素相乘：

$$z = [z_1, z_2], \quad h = (z_1 \sigma(z_1)) \odot z_2$$

```
import torch.nn.functional as F
projection = nn.Linear(100, 200)
z1, z2 = projection(x).chunk(2, dim=-1) # x: [B,100]
h = F.silu(z1) * z2                     # h: [B,100]
```

前層輸出要加倍，分割後才得到需要的 100 維表示。

## BatchNorm：先算批次統計，再學尺度

---

對一個特徵，批次內有  $m$  筆值：

$$\mu_B = \frac{1}{m} \sum_i x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_i (x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad z_i = \gamma \hat{x}_i + \beta$$

$\mu_B, \sigma_B^2$  來自本批資料； $\gamma, \beta$  透過梯度學習。

## BatchNorm 手算：四筆資料的一個特徵

---

本批輸入  $[2, 4, 4, 6]$ ，先忽略很小的  $\epsilon$ 。

1. 算出平均數與變異數，分母使用 4。
2. 計算四個標準化值。
3. 令  $\gamma = 2, \beta = 1$ ，計算最終輸出。

這四個值是「四筆樣本的同一直線」，每一列代表一個人。

## BatchNorm 的訓練與推論

| 狀態                         | 正規化統計來源                    | 是否更新移動統計？ |
|----------------------------|----------------------------|-----------|
| <code>model.train()</code> | 當前批次                       | 是         |
| <code>model.eval()</code>  | 已保存的 running mean／variance | 否         |

PyTorch 的 BN momentum  $m$  表示新資料權重：

$$\mu_{run,new} = (1 - m)\mu_{run,old} + m\mu_B$$

小批次的統計可能不穩定；`eval()` 不會刪除已學到的  $\gamma, \beta$ 。

## PyTorch : Linear 、 BatchNorm 、 ReLU

---

```
model_bn = nn.Sequential(  
    nn.Flatten(),  
    nn.Linear(784, 100, bias=False),  
    nn.BatchNorm1d(100),  
    nn.ReLU(),  
    nn.Linear(100, 10)  
) .to(device)
```

本例在激活前放 BN，線性層省略重複偏置。輸入先完成前處理。

## BN 的統計軸：向量與影像

| 輸入        | 模組             | 每組統計涵蓋哪些值？                     |
|-----------|----------------|--------------------------------|
| [B,C]     | BatchNorm1d(C) | 同一特徵的 B 筆樣本                    |
| [B,C,L]   | BatchNorm1d(C) | 同一通道的 $B \times L$ 個值          |
| [B,C,H,W] | BatchNorm2d(C) | 同一通道的 $B \times H \times W$ 個值 |

BN 的可學參數是每通道一個  $\gamma$  與一個  $\beta$ 。

影像不必先攤平才能使用正規化，模組要配合資料形狀。

## LayerNorm：每筆資料自己算統計

---

對一筆樣本的特徵向量  $h = [h_1, \dots, h_d]$ ：

$$\mu = \frac{1}{d} \sum_j h_j, \quad \sigma^2 = \frac{1}{d} \sum_j (h_j - \mu)^2$$

$$\text{LN}(h)_j = \gamma_j \frac{h_j - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_j$$

`nn.LayerNorm(d)` 正規化最後一個維度，訓練與推論都用本筆統計。

## LayerNorm 手算：一筆資料的四個特徵

---

LayerNorm 手算例： $h = [2, 4, 4, 6]$ 。

$\mu = 4$ ， $\sigma^2 = 2$ ，初始  $\gamma_j = 1, \beta_j = 0$ ：

$$\text{LN}(h) \approx [-1.4142, 0, 0, 1.4142]$$

與剛才 BN 使用相同數字，但這次是一筆樣本的四個特徵。

若批次裡加入另一筆樣本，這筆的 LN 結果會改變嗎？

## BatchNorm 與 LayerNorm 比較

| 比較項目   | BN (以 [B,D] 為例) | LN (以 LayerNorm(D) 為例) |
|--------|-----------------|------------------------|
| 統計方向   | 每一欄跨樣本          | 每一列跨特徵                 |
| 依賴其他樣本 | 訓練時會            | 不會                     |
| 推論統計   | 通常用移動估計         | 即時計算                   |
| 小批次限制  | 統計可能不穩定         | 不依賴批次大小                |
| 常見情境   | CNN／批次訓練        | 序列、Transformer等        |

兩者都可學縮放和平移，需依架構與資料驗證選擇。

## 梯度過大時，應在哪一步裁剪？

---

一個訓練步驟可寫成：

```
zero_grad → forward → loss → backward → clip_grad_norm_ → optimizer.step
```

裁剪要在 `backward()` 產生梯度後、參數更新前。若使用混合精度與梯度縮放，須先解除縮放，再依真實梯度範數裁剪。

## 梯度裁剪：限制一次更新的梯度

---

按範數裁剪：

$$g_{clip} = g \min \left( 1, \frac{c}{\|g\|_2} \right)$$

例如  $g = [3, 4]$ 、 $c = 1$ ，範數為 5，裁剪後為  $[0.6, 0.8]$ 。

按值裁剪至  $[-1, 1]$  則得到  $[1, 1]$ ，可能改變方向。

裁剪抑制過大梯度，不能把消失的梯度放大回來。

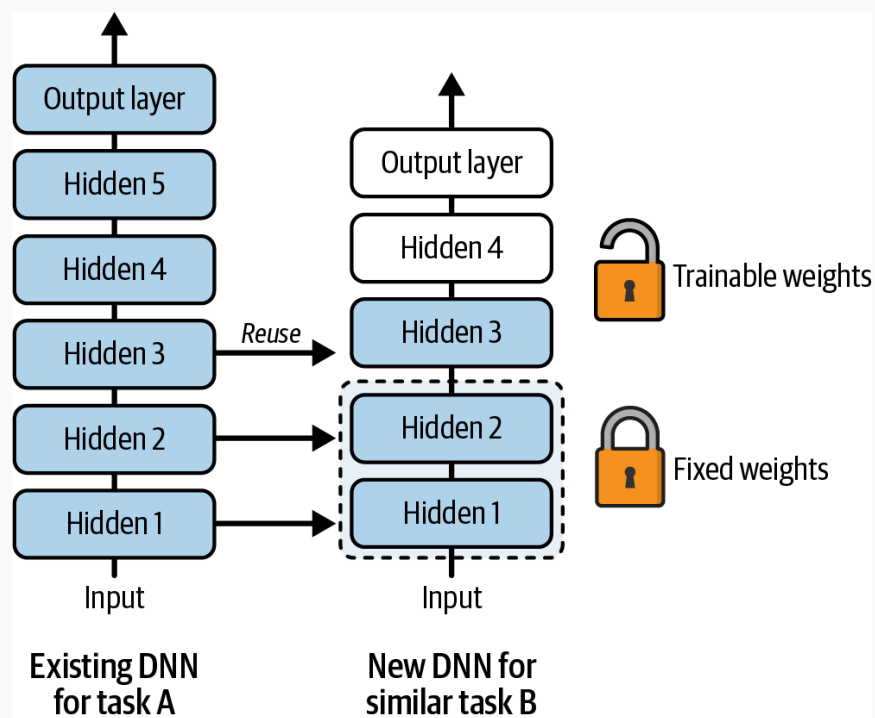
## 梯度裁剪放在 backward 與 step 之間

---

```
optimizer.zero_grad(set_to_none=True)
logits = model(X_batch)
loss = loss_fn(logits, y_batch)
loss.backward()
grad_norm = nn.utils.clip_grad_norm_(
    model.parameters(), max_norm=1.0)
optimizer.step()
```

回傳值可記錄裁剪前的總範數；頻繁裁剪時仍需檢查學習率與資料。

## 圖 11-5：重用已學到的特徵



重用較底層的代表，替換符合新任務的輸出層，再逐步微調。

## 遷移學習：凍結與解凍的順序

| 階段    | 重用層           | 新輸出層 | 原因           |
|-------|---------------|------|--------------|
| 先學新任務 | 凍結            | 訓練   | 避免隨機輸出層破壞原特徵 |
| 局部微調  | 解凍靠近輸出的部分     | 訓練   | 讓高階特徵適應新資料   |
| 再評估   | 視資料量與驗證結果解凍更多 | 訓練   | 兼顧適應能力與過擬合   |

解凍時通常降低學習率，並確認 optimizer 包含新解凍的參數。

## 書中範例：Fashion MNIST 的任務 A 與 B

| 任務                      | 可用資料                      | 輸出         |
|-------------------------|---------------------------|------------|
| A：先學 8 類服飾              | 不含 T-shirt/top 與 Pullover | 8 個 logits |
| B：區分 T-shirt 與 Pullover | 只有 20 張有標註影像              | 1 個 logit  |

從 A 複製隱藏層，在 B 換成二元分類輸出。

B 的損失使用 `BCEWithLogitsLoss`，標籤需轉成 0／1 浮點數。

## PyTorch：複製骨幹，先只訓練新頭

```
import copy
backbone = copy.deepcopy(model_A[:-1])
for p in backbone.parameters():
    p.requires_grad_(False)
model_B = nn.Sequential(backbone, nn.Linear(100, 1)).to(device)
loss_B = nn.BCEWithLogitsLoss()
optimizer_B = torch.optim.AdamW(
    model_B[1].parameters(), lr=1e-3, weight_decay=1e-4)
# 批次內：logits_B = model_B(X_batch).squeeze(-1)
# loss = loss_B(logits_B, y_binary.float())
```

用深複製保護任務 A 的參數；本例骨幹不含 BN／Dropout。

## PyTorch：解凍後建立分組學習率

---

```
for p in model_B[0].parameters():  
    p.requires_grad_(True)  
optimizer_B = torch.optim.AdamW([  
    {"params": model_B[0].parameters(), "lr": 1e-5},  
    {"params": model_B[1].parameters(), "lr": 1e-4},  
, weight_decay=1e-4])
```

本例讓骨幹慢慢調整，新頭較快適應；數值仍要用驗證集選擇。

重建 optimizer 會重設其內部狀態，這是新的微調階段。

## 書中成效：為何 92.5% 不能直接當保證？

---

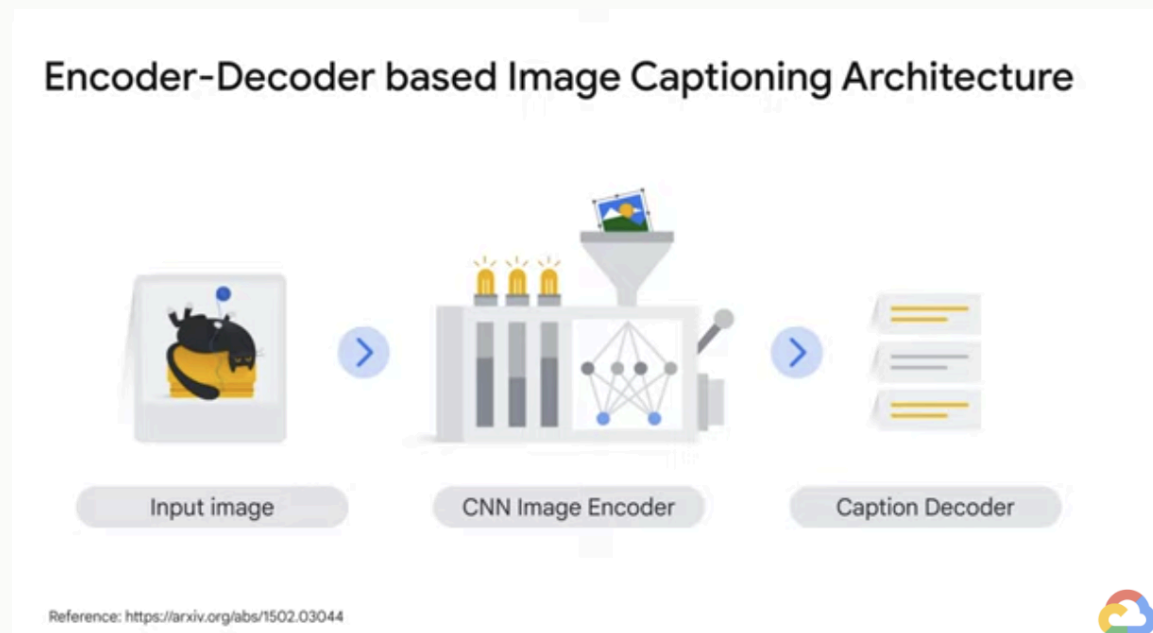
教材展示任務 B 從 **71.6%** 提升至 **92.5%**。

但作者隨後明說：他嘗試了許多設定，刻意挑出改善大的案例。

改變類別或隨機種子後，改善可能縮小、消失，甚至反轉。

課堂比較應固定切分、報告多次結果，並保留從零訓練的基準。

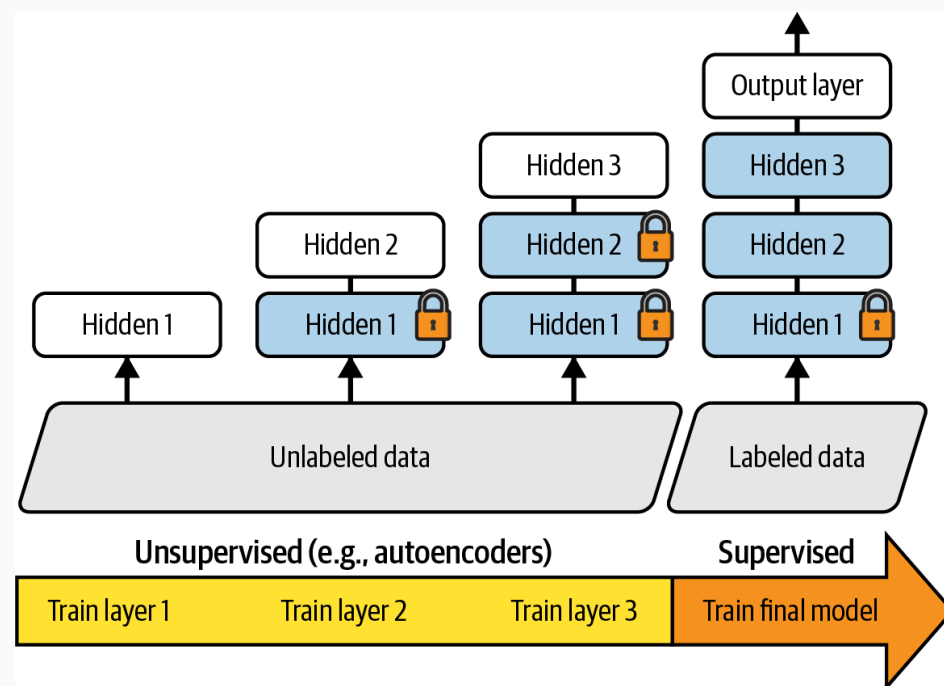
## 遷移學習案例：圖片描述的預訓練 Encoder



先重用 CNN 的影像表示，再讓 Decoder 學習文字描述。

凍結特徵擷取器與微調部分層，是兩個可分別比較的策略。

## 圖 11-6：無監督預訓練的想法



圖示為早期逐層預訓練；今日也可先整體訓練表示，再接上任務輸出。

## 輔助任務與自監督預訓練

---

想做文章分類，但標註很少，可以先用大量文本練習填空：

「這件外套非常\_\_\_\_，適合冬天穿。」

遮住的原字由資料本身提供答案，模型先學語言表示。

之後替換任務輸出，用分類標註微調。預訓練與新任務越相關，越值得比較。

## 小組設計：只有少量服飾標註時

---

已有相關的預訓練分類器，每組設計三種比較：

1. 小網路從零訓練。
2. 凍結預訓練骨幹，只訓練新頭。
3. 在第二組基礎上解凍，使用較小學習率。

提交：固定切分、各組可訓練參數、比較指標與失敗判準。

離線組：用矩陣形狀和訓練步驟說明三種策略，不需執行訓練。

## 離堂檢核：方法與問題要配對

---

1. He 初始化主要想維持什麼？能否保證整段訓練穩定？
2. 為何同一組  $[2, 4, 4, 6]$  可以是 BN 或 LN 的例子？
3. 凍結參數與呼叫 `eval()` 有何不同？
4. 裁剪梯度為何要放在 `backward()` 之後？

課後：書本第 11 章練習 1-4；下一份接續最佳化與正則化。

## Notebook 導讀：可執行的機制與驗收

| 實驗         | 驗收方式                 |
|------------|----------------------|
| 激活與初始化     | 曲線、訊號尺度、SwiGLU 形狀    |
| BN／LN      | 統計軸、均值、推論使用的統計       |
| 凍結再解凍      | 骨幹權重先不變，再以較小學習率更新    |
| 預訓練與圖片描述介面 | 無標籤重建；影像表示接 token 序列 |

Notebook 以 NumPy 驗算；PyTorch／Keras 頁是框架讀碼對照。  
合成任務與形狀檢查不代表重現 Fashion MNIST 或圖片描述品質。